

Koncepcja i implementacja systemu zarządzania pokojami w biurze

(Concept and implementation of office room management system)

Aleksander Uniatowicz

Praca inżynierska

Promotor: dr inż. Leszek Grocholski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

1 września 2022

Streszczenie

W pracy opisuję koncept i implementację systemu służącego do zarządzania pokojami. Przedstawiam rozwiązania dostępne na rynku, końcową aplikację oraz opowiadam o najciekawszych elementach implementacji.

In this paper, I describe concept and implementation of room management system. I show alternative systems, available on the market. I show completed application and most interesting parts of implementation.

Spis treści

1. Wprowadzenie	7
2. Cel i Zakres Pracy	9
3. Przegląd alternatywnych rozwiązań	11
3.1. Alternatywne rozwiązania	11
3.1.1. Google Workspace	11
3.1.2. Teem	12
4. Program	13
4.1. Uruchamianie aplikacji	13
4.2. Skrypt wypełniający pokoje	14
4.3. Działanie programu	14
4.3.1. Strona główna	14
4.3.2. Logowanie	16
4.3.3. Widok pokoju	17
4.3.4. Kod QR	18
4.3.5. Dodawanie wydarzeń	19
4.3.6. Szczegóły wydarzenia	22
4.3.7. Administracja	23
5. Szczegóły implementacji	25
5.1. Użyte technologie	25
5.1.1. Front-end	25
5.1.2. Back-end	26

5.1.3. Devops	27
5.1.4. Jakość kodu	27
5.2. Szczegóły implementacyjne	27
6. Dalszy rozwój aplikacji	31
6.1. Co zrobiłbym inaczej	31
Bibliografia	33

Rozdział 1.

Wprowadzenie

Zarządzanie salami w biurach nie jest łatwym zadaniem. W firmie Nokia jest to organizowane poprzez wysyłanie specjalnych maili na skrzynki pokoi. W odpowiedzi dostaje się informację, czy udało się zarezerwować pokój.

Nie jest to przyjazne użytkownikowi rozwiązanie, ponieważ gdy podejdziemy do salki bez rezerwacji, nie wiemy czy możemy w niej usiąść czy za 10 minut przyjdzie ktoś z rezerwacją i nas z niej wyrzuci.

Używany system rezerwacji był inspiracją do tematu projektu realizowanego pod opieką firmy w ramach programu Innovative Projects by Nokia.

Tematem projektu jest przygotowanie systemu do zarządzania pokojami, oferującego wygodny interfejs do przeglądania dostępnych sal oraz możliwość szybkiego sprawdzania zajętości sali przed którą stoimy.

Rozdział 2.

Cel i Zakres Pracy

Celem pracy jest opracowanie koncepcji i implementacja systemu pozwalającego na sprawne zarządzanie pokojami. W szczególności powinna być możliwość: szybkiego sprawdzenia dostępności pokoju za pomocą skanu kodu QR, dodawania wydarzeń, integracji z kalendarzami. Aplikacja powinna także proponować alternatywne pokoje, jeżeli wybrany przez nas jest już zajęty.

Projekt składa się z osobnych aplikacji: back-endu dostarczającego API oraz front-endu.

Rozdział 3.

Przegląd alternatywnych rozwiązań

Istnieje wiele rozwiązań do zarządzania pokojami. Oferują one wysoki stopień integracji oraz wiele funkcjonalności takich jak: integracja z usługami katalogowymi (np. Microsoft Active Directory), aplikacjami do zarządzania kalendarzami, systemami wideokonferencyjnymi oraz fizycznymi urządzeniami do rezerwacji.

3.1. Alternatywne rozwiązania

3.1.1. Google Workspace

Google Workspace oferuje wbudowaną podstawową obsługę pokoi - możemy tworzyć pokoje oraz organizować w nich spotkania. Rezerwacjami można zarządzać z ekranu kalendarza.

Rozwiązanie to jest bardzo podstawowe. Sale są traktowane jak kalendarz Google, a spotkania jako wydarzenia w tym kalendarzu. Zaletami tego rozwiązania jest prostota konfiguracji oraz bycie częścią większego pakietu.

Wadą tego rozwiązania jest mała liczba funkcji oraz konieczność korzystania z usług Google przez naszą firmę, co w przypadku chęci korzystania tylko z jednej funkcjonalności, jest nieopłacalne (koszt pakietu to od 6 do 18 dolarów miesięcznie ale pakiety ze standardowego cennika, są dostępne dla maksymalnie 300 pracowników). Warto także dodać, że to rozwiązanie będzie wygodne tylko gdy pracownicy będą korzystać z kalendarza Google.

System ten jest często integrowany z innymi rozwiązaniami (w przypadku gdy nasza firma korzysta z tego pakietu), przedstawionymi niżej. Pozwala to zachować wygodę rezerwowania z kalendarza, jednocześnie dając możliwość rozszerzania funkcjonalności.

3.1.2. Teem

Teem oferuje kompleksowe rozwiązanie zawierające aplikacje do zarządzania pokojami, integrację z Google Apps, Office 365 oraz Outlook. Z Teem korzysta wiele dużych firm takich jak HP, LinkedIn oraz Allegro. Firma udostępnia także aplikację na iPady.

Na przykładzie firmy Allegro, Teem był zintegrowany z kalendarzem (Google Workspace). Pozwalało to rezerwować sale przy tworzeniu spotkań. Salki konferencyjne były wyposażone w iPady przy wejściu, gdzie można było sprawdzić czy pokój jest dostępny oraz zarezerwować go "Ad hoc" na spotkania, które wcześniej nie były zaplanowane. Przed początkiem spotkania było konieczne potwierdzenie obecności, co pozwalało unikać niepotrzebnych rezerwacji sali. W przypadku końca spotkania przed przewidzianą godziną, można było zwolnić salę z tabletu przy drzwiach.

Wadą tego rozwiązania jest koszt. Licencja na oprogramowanie, przy standardowym cenniku, kosztuje rocznie 150-250 euro za pokój. Aby wykorzystywać w pełni możliwości oprogramowania, należy wyposażyć salki w tablety, a w przypadku firmy posiadającej setki sal, jest to bardzo duży koszt.

Rozdział 4.

Program

Końcowy program nazwany został Checkroom i można go znaleźć pod linkiem <https://checkroom.herokuapp.com/>. Z uwagi na oszczędzanie zasobów, aplikacja automatycznie "zasypia" po godzinie od ostatniego wejścia. Pierwsze wejście po uśpieniu, wymaga wybudzenia aplikacji. Dzieje się to automatycznie po wejściu na stronę, ale trwa nawet do kilku minut. Kod dostępny jest na platformie github pod linkiem <https://github.com/nokia-wroclaw/innovativeproject-check-room>.

4.1. Uruchamianie aplikacji

Aby stworzyć własną instancję aplikacji, wystarczy zbudować obraz docker za pomocą polecenia `docker build ..`. Dzięki użyciu docker compose, obie aplikacje uruchamiają się jednym poleceniem.

Back-end do działania wymaga zmiennych środowiskowych, składa się na nie:

- `GOOGLE_API_CREDENTIALS`, `GOOGLE_CLIENT_SECRET` które należy wygenerować za pomocą konsoli Google Cloud;
- `GOOGLE_API_TOKEN` które można uzyskać za pomocą dostarczonego z kodem kreatora;
- `MONGO_URL` w którym powinien znajdować się link instancji MongoDB z której chcemy aby aplikacja korzystała.

Front-end wymaga id klienta do logowania Google. Należy je wygenerować w konsoli Google Cloud następnie uzupełnić w pliku `constants.js`.

4.2. Skrypt wypełniający pokoje

Dla ułatwienia testowania i prezentowania aplikacji, powstał skrypt wypełniający pokoje wydarzeniami, uruchamia się go poleceniem

```
npm run seed-rooms
```

Należy skorzystać z opcji polecenia. `--days` decyduje ile dni w przód wypełniamy. `--room` pozwala wybrać pokój ("`all`" pozwala wypełnić wszystkie pokoje). `--eventsPerDay` ustawia maksymalną liczbę wydarzeń dziennie.

Skrypt ten wypełnia pokoje wydarzeniami, rozpoczynającymi się między godzinami 7 a 15 (początki wydarzeń zawsze są o pełnej godzinie lub 30 minut po pełnej godzinie) oraz trwającymi między 1 a 4 godzin (tylko pełne godziny). Skrypt wypełnia pokoje od dnia uruchomienia włącznie przez ustawioną liczbę dni. Instancja testowa została uzupełniona wydarzeniami na sierpień i wrzesień 2022.

Uruchomienie skryptu wymaga uzupełnienia zmiennych środowiskowych, analogicznie do back-endu. Działanie jest dość wolne, ponieważ wykonujemy działania jedno po drugim, zamiast robić to równolegle. Nie zostało to zoptymalizowane, aby nie powodować problemów z rate limitingiem oraz nie uznaję go za konieczne, bo skrypt jest uruchamiany tylko sporadycznie. Uruchomienie skryptu dla jednego dnia na 8 pokojach i 3 wydarzeniach dziennie zajmuje 10 sekund. Jeżeli wylosowane wydarzenie, nakłada się z innym jest ono odrzucane - z tego powodu może zostać dodanych mniej wydarzeń niż ustawiono w `--eventsPerDay`, jest to działanie oczekiwane.

4.3. Działanie programu

Aplikacja z punktu widzenia użytkownika działa jako strona internetowa. Każda osoba, nawet bez logowania, ma dostęp do przeglądania pokoi i wydarzeń. Nie jest to problemem bezpieczeństwa, gdyż w docelowym środowisku, aplikacja byłaby dostępna tylko z sieci wewnętrznej.

Aplikacja jest responsywna, to znaczy przystosowana do korzystania zarówno na urządzeniach mobilnych jak i komputerach czy tabletach.

4.3.1. Strona główna

Po wejściu na stronę, widzimy ekran z listą pomieszczeń i możliwością filtrowania.

Room name	Seats	Building	Projector	Whiteboard
Room 53				
Under construction		Large Building, 4th floor		
Basement				
Developers favourite place.		Large Building, -1st floor		
Room of luck				
Good for meeting with investors		Unknown location		
Big Room				
Good for creating EDM		Unknown location		
Conference hall				
Usually unused but very big		Unknown location		
Room 495				
Gaming room		Unknown location		
Room 11				
A small room		Large Building, 1st floor		
Library				
Currently storing 300 copies of Effective C++		Large Building, 2nd floor		
Empty room				
this room has no events initially		Small Building, 1st floor		

Rysunek 4.1: Strona główna

Filtrowanie pozwala nam na wyszukiwanie po nazwie pokoju, minimalnej liczbie miejsc, pozostałych parametrach (posiadanie projektora i tablicy), oraz budynku i piętrze.

Room name: Room name Seats: Seats

Building: Building

- ☐ Large Building
- ☐ Floor -1
- ☐ Floor 1
- ☐ Floor 2
- ☐ Floor 4

Rysunek 4.2: Filtrowanie po piętrze oferuje wygodną rozwijaną listę z zagnieżdżeniami - pozwala to łatwo wybrać cały budynek lub tylko poszczególne piętra.

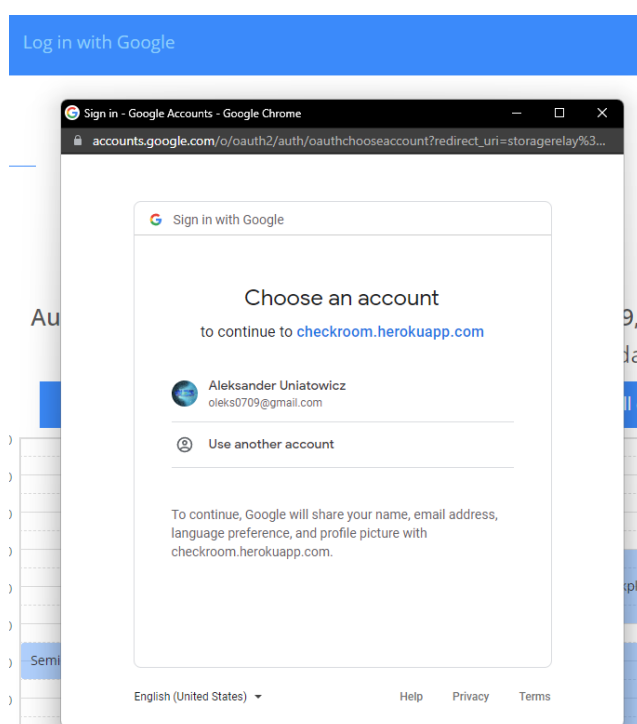
Pod okienkiem filtrowania widzimy listę pokoi.



Rysunek 4.3: Każdy kafelek pokoju zawiera podstawowe informacje o nim, takie jak nazwę, opis, lokalizację i parametry.

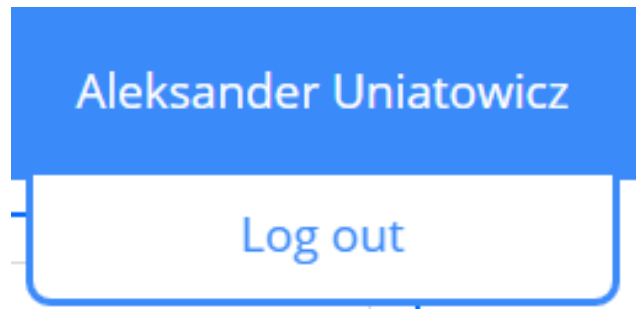
4.3.2. Logowanie

Na pasku widzimy guzik "Log in with Google" pozwalający na logowanie za pomocą konta Google.



Rysunek 4.4: Po kliknięciu guzika zostanie nam wyświetlone okno logowania.

Po zalogowaniu zobaczymy swoje imię i nazwisko (ustawione na koncie Google).

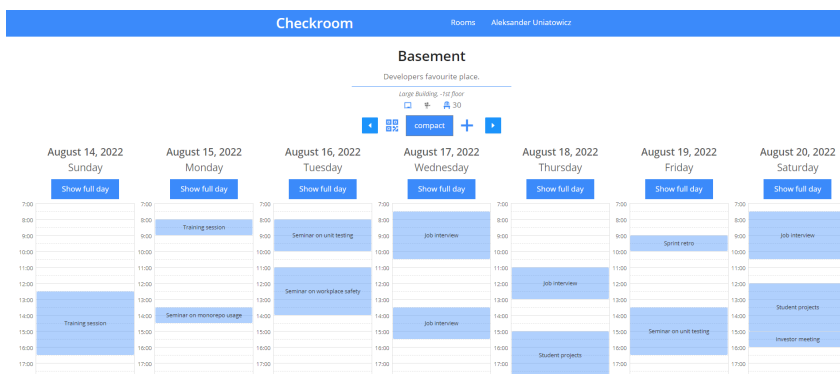


Rysunek 4.5: Po kliknięciu w swoje dane, mamy możliwość wylogowania się.

Domyślnie, każdy nowy użytkownik jest gościem, gość ma takie same uprawnienia jak osoba niezalogowana. Administrator może zweryfikować gościa i nadać mu uprawnienia użytkownika. Użytkownik może dodawać wydarzenia oraz edytować te, których jest twórcą.

4.3.3. Widok pokoju

Na widoku pokoju możemy zobaczyć informacje o pokoju oraz kalendarz z wydarzeniami.

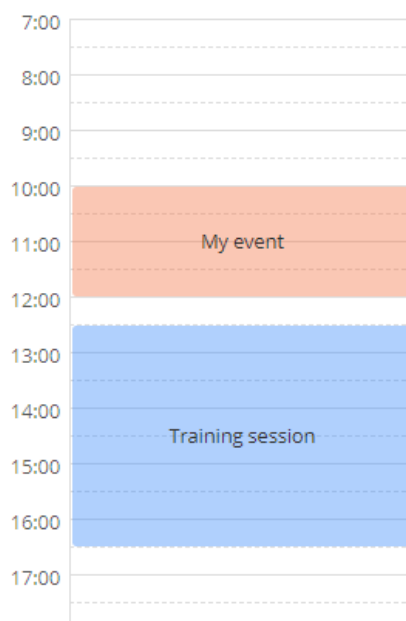


Rysunek 4.6: Widok pokoju



Rysunek 4.7: Użytkownik (po prawej) widzi guzik dodawania wydarzeń, gość (po lewej) go nie widzi.

Na widoku pokoju możemy przeglądać wydarzenia.



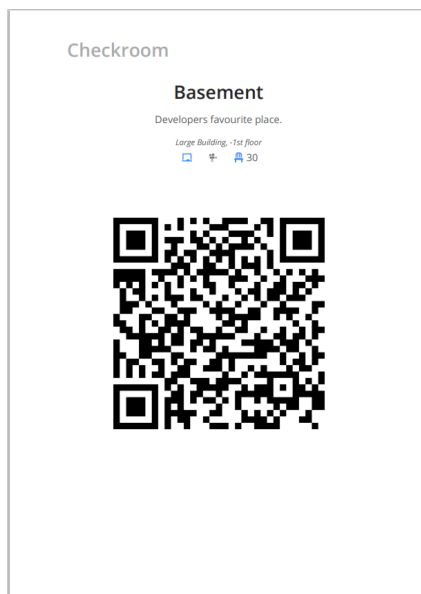
Rysunek 4.8: Kolorem niebieskim oznaczone są wydarzenia utworzone przez innych a pomarańczowym nasze.

Mamy jeszcze dostępny widok kompaktowy - pozwala on zobaczyć 7 dni na telefonie. W trybie tym widzimy tylko jeden rząd godzin oraz wszystkie dni są skompresowane, tak aby mieściły się w jednym wierszu.

Z widoku pokoju możemy także przejść do strony z QR kodem.

4.3.4. Kod QR

Strona z QR kodem jest przewidziana do druku. Widzimy na niej podstawowe informacje o pokoju, tak samo jak na poprzednim widoku, oraz kod QR. Kod możemy wydrukować za pomocą funkcji "drukuj" w przeglądarce (skrót ctrl/cmd+P).

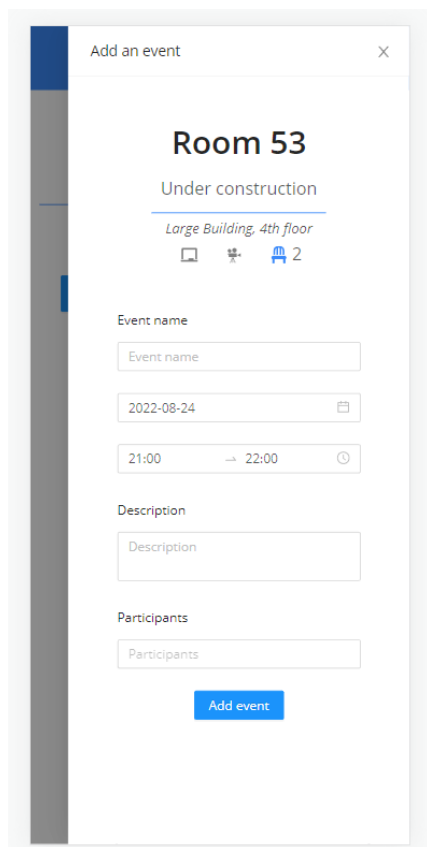


Rysunek 4.9: Widok wydruku

Po wydrukowaniu, możemy taką kartkę powiesić przed salą. Dzięki temu osoby przed nią będą mogły szybko sprawdzić dostępność pokoju. Kod kieruje bezpośrednio do strony pokoju.

4.3.5. Dodawanie wydarzeń

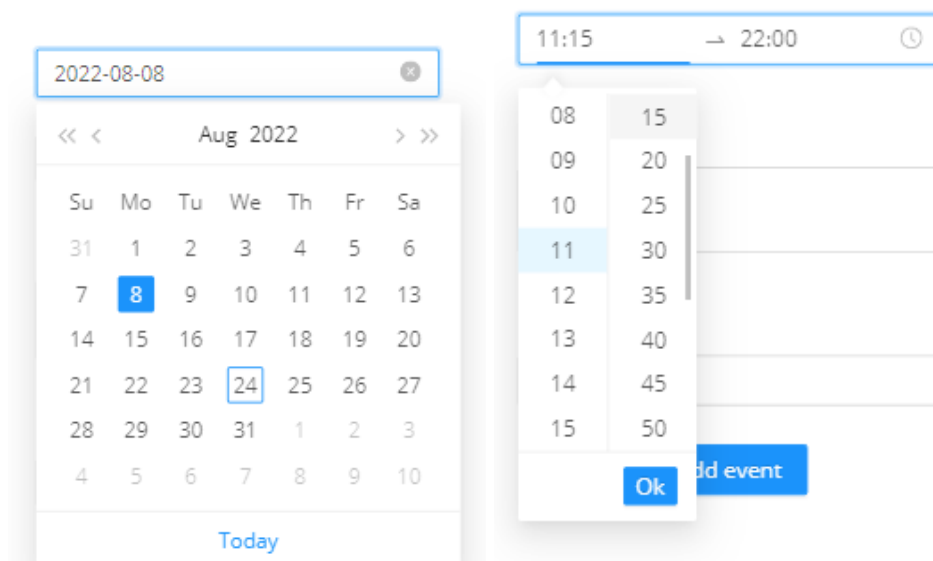
Po kliknięciu plusa w ekranie pokoju, z prawej strony wysuwa się szuflada z formularzem dodawania pokoju.



The screenshot shows a mobile application interface for adding an event. At the top, there's a header bar with the text 'Add an event' and a close button (X). Below this, the room name 'Room 53' is displayed in a large, bold font. Underneath, it says 'Under construction' in a smaller font, followed by 'Large Building, 4th floor' in an even smaller font. There are three icons: a calendar, a location pin, and a group of people icon with the number '2' next to it. Below these, there are three input fields: 'Event name' (with a placeholder 'Event name'), a date field (with the value '2022-08-24' and a calendar icon), and a time field (with the value '21:00' and '22:00' separated by a double arrow, and a clock icon). Below the time field is a 'Description' field with a placeholder 'Description'. At the bottom, there is a 'Participants' field with a placeholder 'Participants'. A blue button labeled 'Add event' is located at the very bottom of the form.

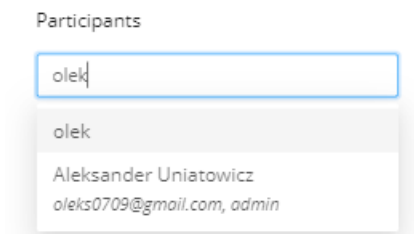
Rysunek 4.10: Formularz dodawania pokoju

Możemy tam podać nazwę dodawanego wydarzenia, jest to pole obowiązkowe. Możemy także ustawić datę i godziny rozpoczęcia i zakończenia dodawanego przez nas spotkania. Domyślnie te pola są uzupełnione bieżącym dniem oraz najbliższym początkiem godziny jako datą rozpoczęcia i godziną następującą po niej jako datą zakończenia.



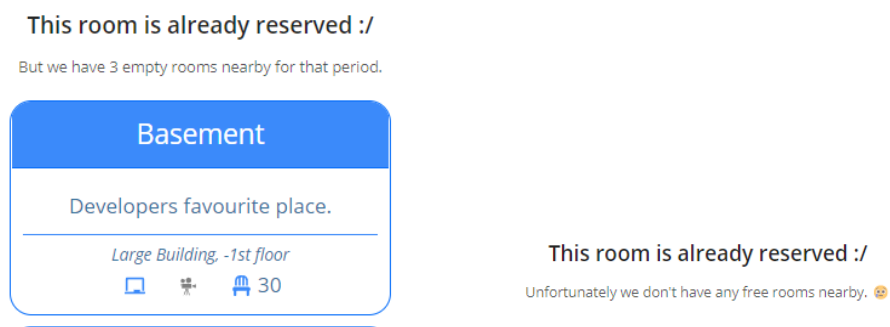
Rysunek 4.11: Datę i godziny uzupełnia się z dedykowanych komponentów.

Mamy też 2 opcjonalne pola: opis wydarzenia oraz listę uczestników. Do tych uczestników zostanie wysłane zaproszenie mailowe.



Rysunek 4.12: Dodawanie gości jest interaktywne - możemy wybrać ich z listy użytkowników aplikacji (lub wpisać adres e-mail ręcznie, jeżeli użytkownika nie ma na liście)

Gdy klikniemy dodaj wydarzenie, mogą stać się dwie rzeczy. Może nam udać się dodać wydarzenie - wtedy szuflada się zamknie, a dane o wydarzeniach ponownie się załadują - pokazując nam nasze nowe spotkanie. Jeżeli jednak nasze spotkanie koliduje z istniejącymi już rezerwacjami, dostaniemy komunikat o braku możliwości rezerwacji oraz zostaną nam wyświetlone pokoje w tym budynku (lub stosowny komunikat w przypadku jego braku).

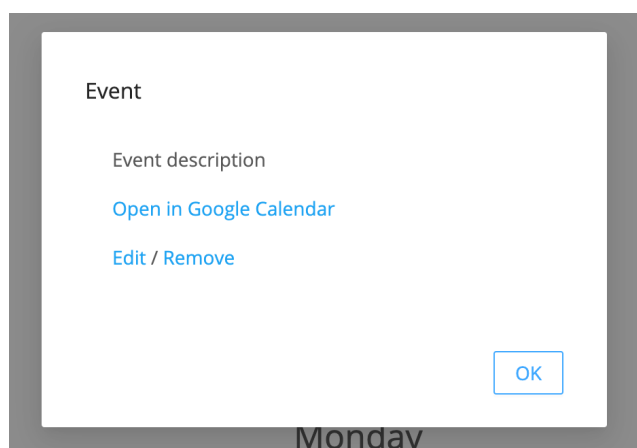


Rysunek 4.13: Komunikaty

Po wybraniu pokoju zastępczego, zostajemy przekierowani do niego, z wypełnionym formularzem.

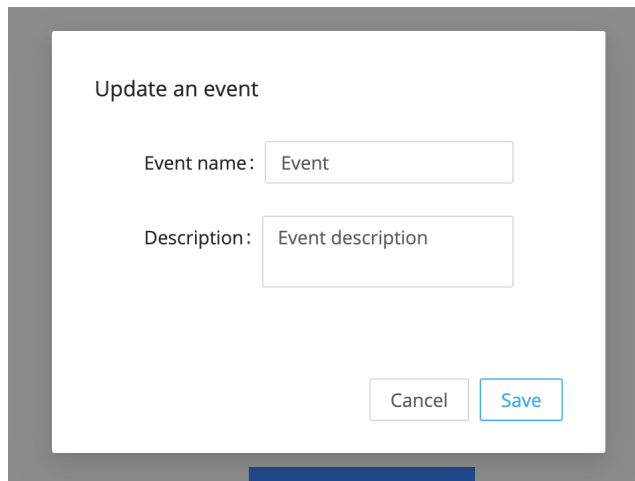
4.3.6. Szczegóły wydarzenia

Po wybraniu wydarzenia, możemy zobaczyć modal z jego szczegółami.



Rysunek 4.14: Szczegóły wydarzenia. W przypadku wydarzenia nie należącego do nas, guziki edycji będą niedostępne.

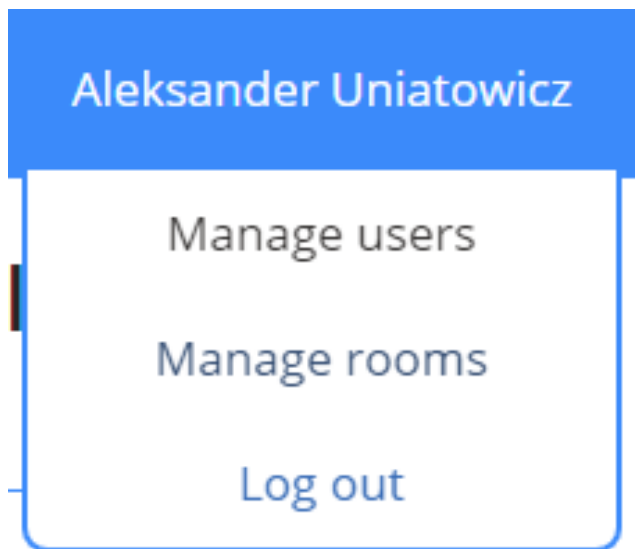
Jeżeli wydarzenie należy do nas, mamy możliwość jego usunięcia lub edytowania detali.



Rysunek 4.15: Ekran edycji wydarzenia.

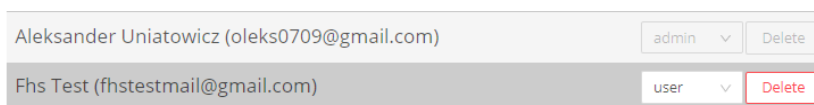
4.3.7. Administracja

Aplikacja umożliwia zarządzanie pokojami i użytkownikami z wygodnym GUI. Administratorzy mają dodatkowe opcje w panelu użytkownika.



Rysunek 4.16: Panel u administratora

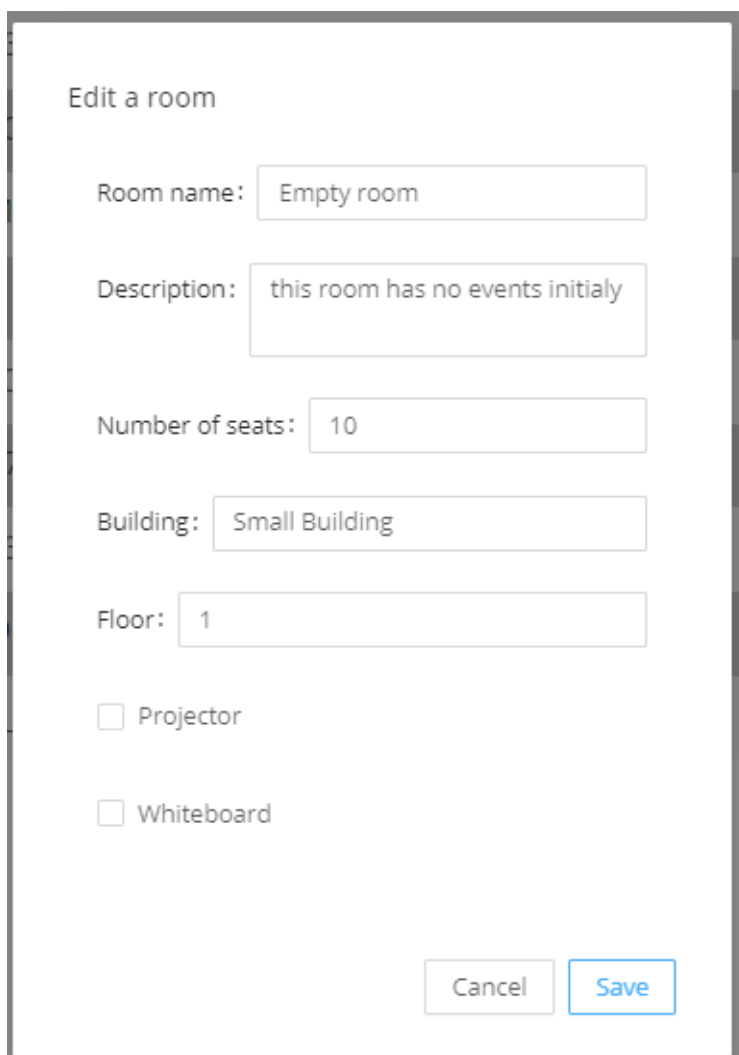
Możemy zarządzać użytkownikami - zmieniać ich rolę oraz usuwać. Administrator nie może edytować własnych uprawnień ze względów bezpieczeństwa.



Rysunek 4.17: Panel edycji użytkownika

Możemy także zarządzać pokojami. Na ekranie widzimy listę pokoi oraz mamy możliwość usunięcia ich oraz uruchomienia panelu edycji. Pod listą znajduje się guzik dodający pokój, który otwiera modal w którym możemy wypełnić parametry pokoju.

Każdy pokój ma także guzik, który pozwala na edycje wszystkich parametrów utworzonego już pokoju.



The image shows a modal dialog box titled "Edit a room". It contains several form fields for editing room information:

- Room name:** A text input field containing "Empty room".
- Description:** A text input field containing "this room has no events initialy".
- Number of seats:** A text input field containing "10".
- Building:** A text input field containing "Small Building".
- Floor:** A text input field containing "1".
- Projector:** A checkbox that is currently unchecked.
- Whiteboard:** A checkbox that is currently unchecked.

At the bottom right of the modal, there are two buttons: "Cancel" and "Save". The "Save" button is highlighted with a blue border.

Rysunek 4.18: Modal edycji/utworzenia pokoju

Rozdział 5.

Szczegóły implementacji

5.1. Użyte technologie

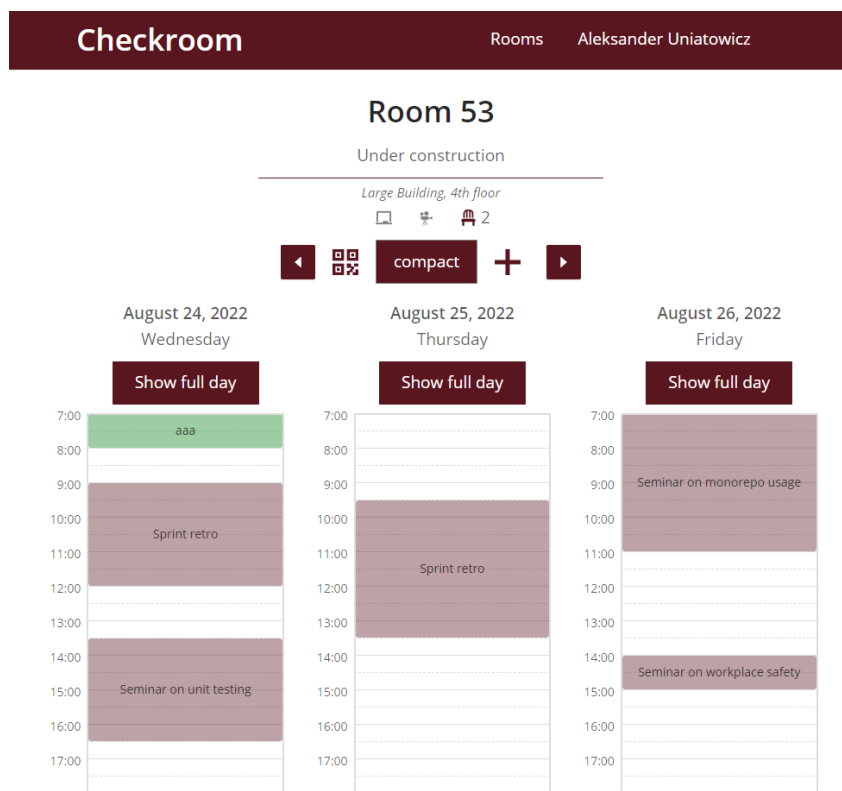
Wymieniam tylko moim zdaniem najciekawsze biblioteki.

5.1.1. Front-end

Front-end aplikacji napisany jest z użyciem biblioteki `React`^{[\[1\]](#)}. `React` jest aktualnie najpopularniejszą biblioteką do tworzenia aplikacji internetowych. Został wybrany ze względu na duże możliwości.

W aplikacji została także wykorzystana biblioteka komponentów `antd`^{[\[2\]](#)}. Moim zdaniem jest to bardzo dobrze napisana biblioteka. Komponenty, które dostarcza, mają dobrze przemyślane API. W wielu miejscach mamy możliwość zmieniania działania komponentów, co pozwala dostosować je do naszych potrzeb. Dzięki temu nie musimy implementować samodzielnie znacznej części interfejsu użytkownika.

Do stylowania aplikacji służy `styled-components`^{[\[3\]](#)}. Biblioteka ta pozwala na tworzenie dynamicznie generowanych i nie powtarzających się klas dla komponentów. W ten sposób unikamy przypadkowego modyfikowania stylów innych części aplikacji poprzez np. pokrywanie się nazw klas. Ciekawym dodatkiem jest biblioteka `polished`^{[\[4\]](#)} - dostarcza ona wiele funkcji pomocniczych do stylów, pozwalając np. na rozjaśnianie koloru lub zmianę jego przeźroczystości. Ułatwia nam to definiowanie kolorów w motywie aplikacji. Dzięki temu jeżeli zdecydujemy się na zmianę koloru, wystarczy zrobić to w jednym miejscu.



Rysunek 5.1: Przykład zmiany głównego koloru na bordowy, a drugoplanowego na zielony - ta zmiana wymagała edycji tylko 2 linijek w kodzie.

5.1.2. Back-end

Backend został wykonany w Node.js[5] za pomocą frameworka Express[6]. Express przez swoją popularność nazywany jest standardowym frameworkiem do pisania serwerów w JavaScriptcie. Ekspres ma wiele zalet, wśród nich można wymienić: deklaratywną obsługę ścieżek, rozszerzalność za pomocą dostarczanych przez społeczność middlewarów, obsługę większości potrzebnych rzeczy, przejrzystą dla programisty.

Do porozumiewania się z API Google, używamy biblioteki `googleapis`[7] a do logowania `google-auth-library`[8].

Jako baza danych do części informacji, służy MongoDB[9], które ze względu na swoją prostotę i prędkość, przez wiele dużych firm jest traktowane jako domyślny wybór, a inne bazy danych są używane tylko gdy znajdzie taka potrzeba biznesowa. Używamy ODM `mongoose`[10], aby móc definiować schematy bazy danych w aplikacji.

5.1.3. Devops

Aplikacja jest wystawiana na Heroku[11]. Zostały stworzone pipeline do automatycznego tworzenia kontenerów Docker[12] kodu po zmianach oraz ich publikacji. Baza danych stoi na MongoDB Atlas[13].

5.1.4. Jakość kodu

Nad jakością kodu czuwa ESLint[14], najpopularniejsze narzędzie do statycznej analizy kodu w JavaScript. Dobraliśmy szeroki zestaw reguł stworzonych przez firmę Airbnb [15]. A o spójność formatowania dbał Prettier[16]. Back-end aplikacji ma także kilka testów jednostkowych napisanych w Jest[17].

Wymienione wyżej narzędzia są uruchamiane w środowisku CI/CD.

5.2. Szczegóły implementacyjne

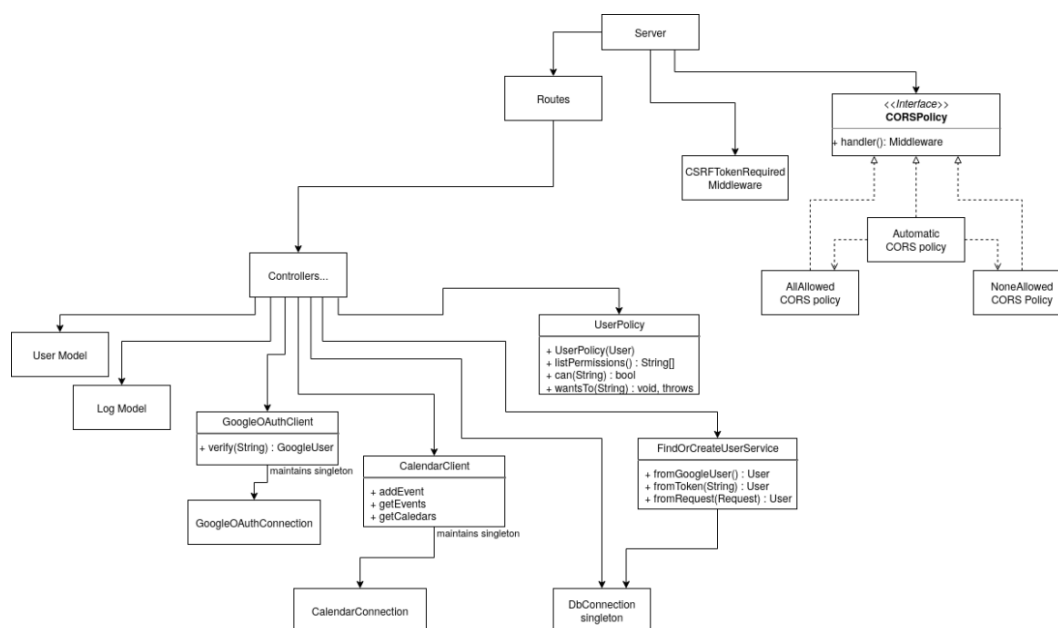
Najciekawszą częścią Back-endu jest wykorzystanie kalendarza Google jako głównej bazy danych. Aplikacja ma swoje konto Google i zarządza stanem poprzez edycję kalendarzy. Pokój jest reprezentowany jako kalendarz, z opisem zawierającym informacje o pokoju. Jako że opis ma maksymalną wielkość, format zapisania tam danych zakłada bardzo krótkie nazwy kluczy w obiekcie JSON. Dla programisty jest to przeźroczyste, dzięki zastosowaniu wzorca `Data Transfer Object`[18]. Za pomocą `Google Api` możemy dodawać nowe pokoje, oraz edytować te już dostępne.

Spotkania dodajemy jako zwykłe wydarzenia kalendarza. Jest to wygodne, ponieważ wszystkie niezbędne pola są już przygotowane. Dodatkowo kalendarz obsługuje nam wysyłanie wiadomości z zaproszeniami.

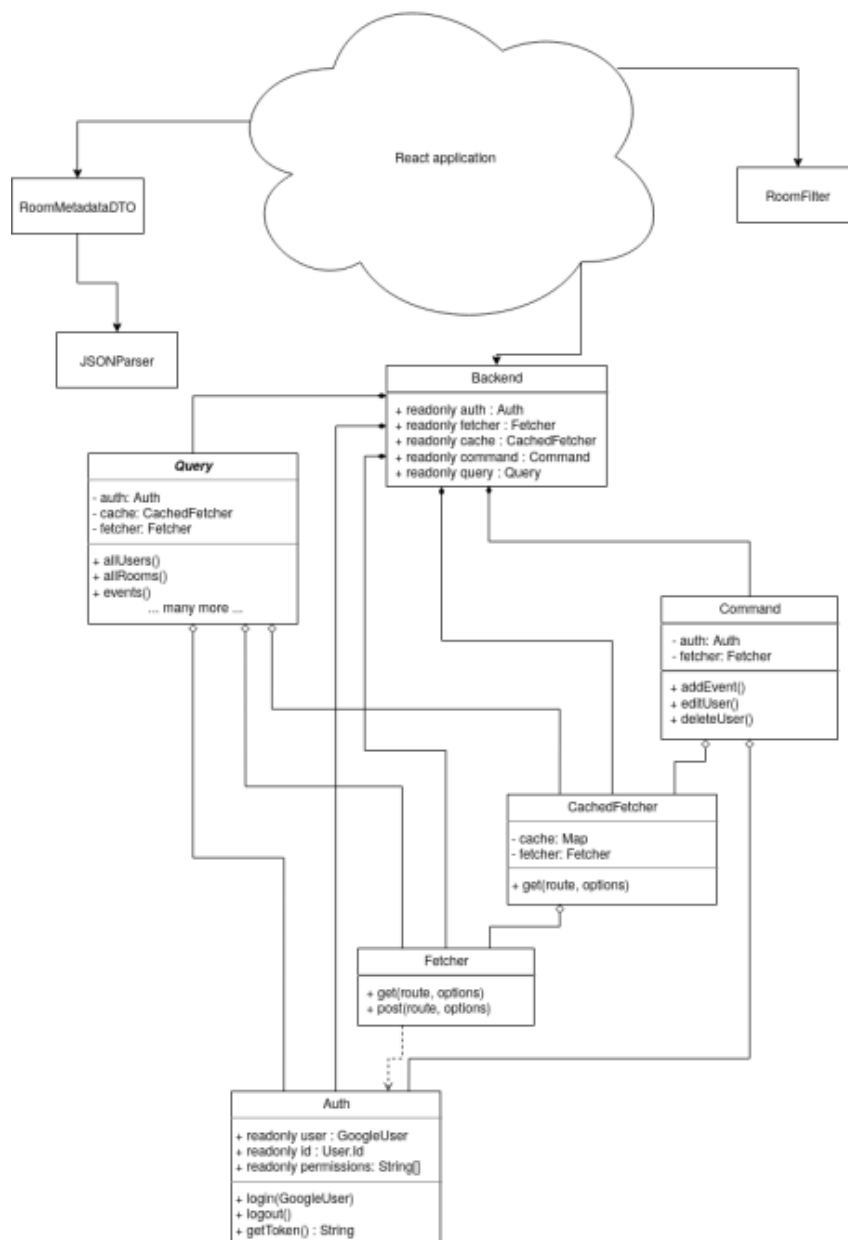
Dodatkowe dane są trzymane w MongoDB, są to dane o właścicielach wydarzeń wymagane do zarządzania uprawnieniami do edycji wydarzenia oraz dane użytkownika.

```
const userSchema = new Schema( {  
  name: { type: String, required: true },  
  email: { type: String, required: true },  
  googleId: { type: String, required: true },  
  type: { type: String, default: 'guest' },  
});  
  
const eventOwnerSchema = new Schema( {  
  userId: { type: String, required: true },  
  googleEventId: { type: String, required: true },  
});
```

Rysunek 5.2: Schematy mongoose dla danych trzymanych w MongoDB

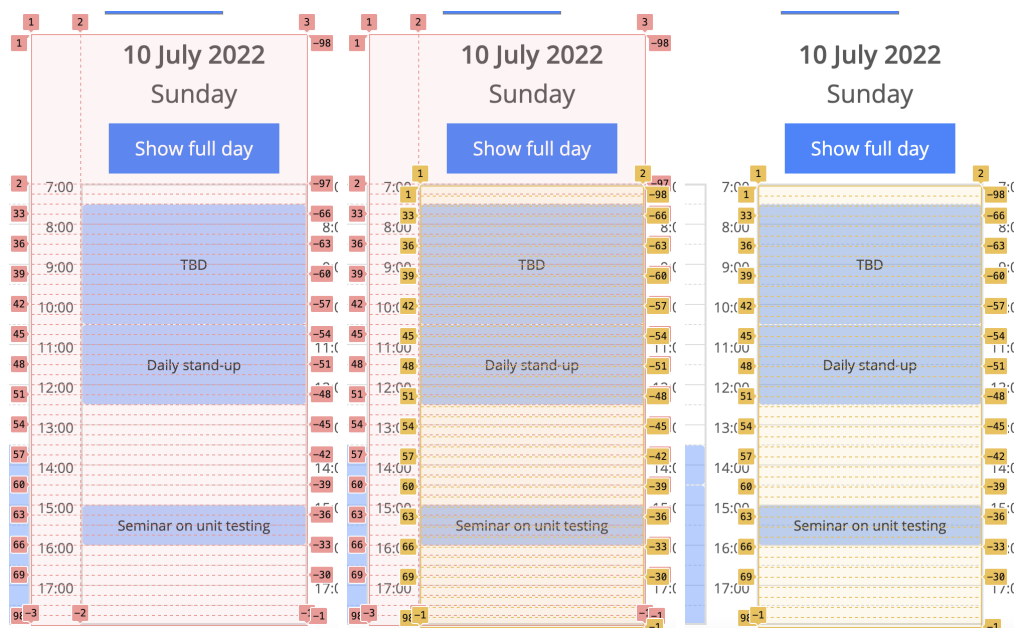


Rysunek 5.3: Diagram klas back-endu



Rysunek 5.4: Diagram klas warstwy łączenia się z danymi Front-endu

Najciekawszym elementem interfejsu użytkownika jest kalendarz. Zastosowana została podwójna siatka (podwójny CSS Grid). Zewnętrzna warstwa służy do pozycjonowania czasu oraz pasków oznaczających godziny i półwki godzin. Wewnętrzna sekcja przechowuje wydarzenia i ma wartości kolumn zostawione niewypełnione, pozwala to na automatyczne przyznawanie przestrzeni, co w przypadku wielu wydarzeń w jednym momencie, podzieli szerokość kolumny między nie. Taka sytuacja nie powinna się zdarzyć, ale warto być na nią przygotowanym.



Rysunek 5.5: Po lewej zewnętrzna warstwa siatki, po prawej wewnętrzna

Rozdział 6.

Dalszy rozwój aplikacji

Aplikacja mogłaby dostać wiele aktualizacji poprawiających wygodę użytkownika, wśród nich są: możliwość szerszej edycji wydarzeń (aktualnie jest to tylko tytuł oraz opis), wydarzenia cykliczne, dokładniejszy system uprawnień (np. możliwość udostępniania tylko określonych pokoi danej osobie).

6.1. Co zrobiłbym inaczej

Robiąc aplikację teraz, zastosowałbym podobne technologie. Jedyną różnicą byłoby zastosowanie `TypeScript`[19]. Pozwala to na stosowanie typów podczas tworzenia aplikacji, co ułatwia pracę oraz pozwala na uniknięcie niektórych błędów. Edytory też lepiej rozumieją kod w `TypeScript`, w związku z tym lepiej podpowiadają składnię. Wadą tej technologii jest dodatkowy narzut czasowy, związany z pisaniem typów, jednak przy nowych aplikacjach nie jest on duży. Problemатyczne są jedynie sytuacje, gdy nasze zależności (lub stary kod) nie mają typów. Wtedy w wielu miejscach musimy omijać typy. Aktualnie rzadko jest to problemem w przypadku bibliotek publicznie dostępnych. Za sprawą projektu `DefinetyTyped`[20], większość popularnych bibliotek otrzymała typy tworzone przez społeczność. Wiele projektów decyduje się także na dodawanie typów do swoich bibliotek, właśnie przez ten projekt (a nie bezpośrednio w repozytorium swojej biblioteki).

W przypadku komercyjnego rozwiązania, nie używałbym też kalendarza Google jako bazy danych. Został on wykorzystany jako proof of concept, który uważam za udany. Mimo że rate limiting jest aktualnie dość wysoki i przy korzystaniu z aplikacji nawet przez wiele osób, są bardzo małe szanse że nas dotknie, to nie mamy gwarancji że się nie zmieni w przyszłości. Prawdopodobnie nawet wtedy dałoby się go obejść, stosując np. różne konta Google na każde kilka pokoi.

Bibliografia

- [1] <https://reactjs.org/>
- [2] <https://ant.design/>
- [3] <https://styled-components.com/>
- [4] <https://polished.js.org/>
- [5] <https://nodejs.org/en/>
- [6] <https://expressjs.com/>
- [7] <https://www.npmjs.com/package/googleapis>
- [8] <https://www.npmjs.com/package/google-auth-library>
- [9] <https://www.mongodb.com/>
- [10] <https://mongoosejs.com/>
- [11] <https://www.heroku.com/>
- [12] <https://www.docker.com/>
- [13] <https://www.mongodb.com/atlas/database>
- [14] <https://eslint.org/>
- [15] <https://github.com/airbnb/javascript>
- [16] <https://prettier.io/>
- [17] <https://jestjs.io/>
- [18] <https://stackabuse.com/data-transfer-object-pattern-in-java-implementation-and-m>
- [19] <https://www.typescriptlang.org/>
- [20] <https://github.com/DefinitelyTyped/DefinitelyTyped>